Dataset with brain atlas A: use the Oasis dataset for the classification of male and female individuals based on structural brain scans

```
import numpy as np
from nilearn import datasets
from nilearn.input_data import NiftiLabelsMasker
from nilearn.image import index_img
import nibabel as nib

brain_data = datasets.fetch_oasis_vbm(n_subjects=100)
yeo = datasets.fetch_atlas_schaefer_2018(n_rois=200)  # this needs to be modified for some tasks
print('%i regions found in this atlas' % (len(np.unique(nib.load(yeo.maps).get_data().ravel())) - 1))

masker = NiftiLabelsMasker(labels_img=yeo.maps, standardize=True,
    memory='nilearn_cache')
input_variables = masker.fit_transform(brain_data.gray_matter_maps)
output_variable = np.array(brain_data.ext_vars.mf == 'F', dtype=int)  # gives 1 for females and 0 for males
```

Task 1- Benchmark model depth. Use sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(10,), activation='relu') to fit deep neural network classifiers (DNNs) based on the lowest resolution data (100 ROIs (n_rois)) using the first 90 subjects as the training set. Fit 5 different DNNs with increasingly deeper latent representations: (10,), (10, 10), (10, 10, 10), (10, 10, 10, 10), and (10, 10, 10, 10, 10), and train/evaluate each of these architectures using different values for alpha (0.0001 / 0.001 / 0.01 / 0.1 / 1.0). Next, evaluate each fitted DNNs model on the unseen last 10 subjects. Please plot the classification accuracy on the training set and the test set (x axis=5 different depths of the DNN architectures, y axis=prediction accuracy, 5 line with 5 colors based on alpha=0.0001 / 0.001 / 0.01 / 0.1 / 1.0; separate plots for train and test).

Task 2- Benchmark model width. Use sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(10,), activation='relu') to fit deep neural network classifiers (DNNs) based on the lowest resolution data (100 ROIs (n_rois)) using the first 90 subjects as the training set. Fit 5 different DNNs with increasingly wide latent representations: (5,), (10), (25), (50), and (100). Next, evaluate each fitted DNNs model on the unseen last 10 subjects. Please plot the classification accuracy on the training set and the test set (x axis=5 different widths of the DNN architectures, y axis=prediction accuracy, 5 line with 5 colors based on alpha=0.0001 / 0.001 / 0.01 / 0.1 / 1.0; separate plots for train and test).

Task 3- Benchmark model with different types of non-linearity. Use sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(10,), activation='relu') to fit deep neural network classifiers (DNNs) based on the lowest resolution data (100 ROIs (n_rois)) using the first 90 subjects as the training set. Fit 5 different DNNs with latent representations based on different kinds of non-linear transformation in the hidden layers of the model: activation='identity', 'logistic', 'tanh', and 'relu'. Next, evaluate each fitted DNNs

model on the unseen last 10% of the structural brain scans. Please plot the classification accuracy on the training set and the test set (x axis=4 different activation functions of the DNN architectures, y axis=prediction accuracy, 5 lines with 5 colors based on alpha=0.0001 / 0.001 / 0.01 / 0.1 / 1.0; separate plots for train and test).

Task 4- Same as task 1. But using alpha=0.0001 throughout. Instead, use random_state=0...99 (random_state of the MLP model) to produce 100 lines from 100 repetitions of the same modeling approach with these different random initalizations.

Task 5- Calibration plot: Like task 3 but using alpha=0.0001 throughout and k-fold k=10 cross-validation on 100 brain regions to produce a calibration plot (requires predict_proba() function, implemented in sklearn) for the four activation functions. Compare these calibration curves to that of LogisticRegression (default parameters) but also plotting its calibration curve based on the same cross-validation scheme

Dataset with brain atlas B: use the Oasis dataset to predict a **continuous outcome** (age) from structural brain scans

```
import numpy as np
from nilearn import datasets
from nilearn.input_data import NiftiLabelsMasker
from nilearn.image import index_img
from sklearn.preprocessing import StandardScaler
import nibabel as nib

brain_data = datasets.fetch_oasis_vbm(n_subjects=100)
crad = datasets.fetch_atlas_craddock_2012()
atlas_nii = index_img(crad['scorr_mean'], 42) # this needs to be modified for some tasks
print('%i regions found in this atlas' % (len(np.unique(atlas_nii.get_data().ravel())) - 1))

masker = NiftiLabelsMasker(labels_img= atlas_nii, standardize=True)
input_variables = masker.fit_transform(brain_data.gray_matter_maps)
output_variable = StandardScaler().fit_transform(brain_data.ext_vars.age[:, None])[:, 0] #
gives subject age on standard units after z-scoring
```

6- analogous to task 1 but predicting continuous subject age (using MLPRegressor instead) and using the lowest resolution of Craddock region atlas resolution (.scorr_mean). Use R2 coefficient of determination instead of classification accuracy, which can be found at *sklearn.metrics.r2_score.*

7- analogous to task 2 but predicting continuous subject age and using the lowest resolution version of Craddock region atlas resolution. Use R2 coefficient of determination instead of classification accuracy.

8- analogous to task 3 but plotting the R2 coefficient instead of accuracy.

9- analogous to task 4

10- Predict subject age using a MLPRegressor and training routine that is analogous to task 5. Groups the subjects into 10 bins based on age (bins of size ([max age] - [min age])/10), then for subjects in each age bin calculate the mean squared error (MSE) between the true and predicted ages, across the 10 folds. Plot the MSE vs age bin (10 age bins on the x-axis, mean MSE across subjects in that age bin on the y-axis). For each age bin there should only be one point (the mean of predicted age MSE across subjects in that age bin).